Abstract

Security misconfigurations are one of the most widespread and actively used threat vectors in the contemporary web applications, yet they are largely under-addressed by generalized tools of vulnerability assessment. Many modern scanners are designed to analyse on a wide-spectrum based on the thousands of Common Vulnerabilities and Exposures (CVE) signatures and will not systematically cover the specific configuration-level faults as defined in **OWASP A05:2021** category. Moreover, the products of these tools often remain confined in the technical lingo, posing a serious communication barrier to non-technical e-commerce stakeholders.

This project proposes MisconfigLab, a dedicated security research infrastructure that is intended to be used in the automated detection, classification and business-relevant reporting of security misconfigurations in e-commerce settings. The platform, created with a Python Flask micro-framework and a robust SQLite relational store, deploys a secure, Role-Based Access Control (RBAC) architecture to ensure strict logical separation between administrative scanning activities and client-facing risk analysis.

The scanning engine provided by MisconfigLab has four specific operational templates: Full **Web Application and Infrastructure, Security Headers Analysis, File and Directory Exposure, and E-commerce Anomalies**. The strategic approach creates very precise results, which are then overlaid to a customized severity map. More importantly, MisconfigLab is unique in comparison with other traditional DAST tools in that it is dedicated to the translation of technical vulnerabilities into some practical, actionable intelligence. It includes a proprietary mathematical algorithm, a Transaction Safety Score. This score ranks vulnerabilities based on the degree of their effect on the e-commerce transaction lifecycle, such that non-technical stakeholders have an easy, graded measure of organizational risk.

The platform was empirically tested against controlled vulnerable targets and was found to have almost **a 93% True Positive Rate regarding critical retail flaws, including missing Content Security Policies and exposed environment files**. The system also completes the vulnerability management lifecycle by automating the creation of formal executive-level security assessment reports, in PDF format, which is directly mapped to the requirements of the **PCI-DSS v4.0 compliance**. MisconfigLab therefore offers a democratized system of scaling the protection of digital retail infrastructure against configuration drift in operation.

Table of Contents

Acknowledgements

Chapter 1: Introduction

1.1 Background to the Research

1.2 The Research Problem

1.3 Aims and Objectives

1.4 Research Questions

1.5 Significance of the Study

Chapter 2: Literature Review

Introduction

Background on E-commerce Platforms

Why are E-commerce Platforms High Risk?

Case Study on Security Misconfiguration as Core Risk

Addressing the problem statement and research gap

Comparative Evaluation of Existing Mitigation Approaches

Conclusion to the Literature Review

Chapter 3: Methodology and Research Method

3.1 System Architecture and Technical Justification

3.2 Backend Processing

3.3 Detection Modules and Scan Template Design

3.4 Ethical and Legal Concerns

3.5 Introduction to System architecture

3.6 Application Routing and Backend Architecture

3.7 Database Schema and ORM Implementation

3.8 Custom Detection Logic and Scanning Engine

3.9 Integration of infrastructure (Nmap Sub processing)

3.10 HTTP Header and File exposure Analysis

3.11 Role-Based Access Control (RBAC)

3.12 Frontend Presentation and Report Generating

3.13 Interactive Dashboards

3.14 PDF Reporting Engine

Chapter 4: Results and Analysis

4.1 Introduction

4.2 Experimental Design and Tests

4.3 Effectiveness in Detection of Vulnerabilities (Quantitative Results)

4.3.1 Client-Side and Header Fuzzing (Template 2)

4.4 Validation of the Algorithmic Risk Scoring Model

4.5 Metrics of Computational Performance and Latency

4.6 Edge Cases and Operational Limitations

Chapter 5: Conclusion and Future Directions

5.1 Synthesis of Project Achievements

5.2 Fulfilment of Research Objectives

5.3 Professional Alignment: OWASP, PCI-DSS and ISO Standards

5.3.1 OWASP Top 10 Integration

5.3.2 PCI-DSS Compliance

5.3.3 Road to ISO/IEC 27001 Certification

5.4 System Limitations and Critical appraisal

5.5 Socio-Technical Impact and Global Retail Resilience

5.5.1 Sustainability of Security Operations

5.6 Future Implementation Roadmap

5.6.1 Phase 1 Venerable AI Integration

5.6.2 Phase 2: Automatic remediation (Auto-Fix)

5.6.3 Phase 3: API and DevSecOps Integration of Enterprise

Appendix A: Professional, Ethical, Legal, and Social (PELS) Analysis

Chapter 1: Introduction

1.1 Background to the Research

The blistering and continuous growth of the web-based e-commerce systems has fundamentally changed the way business is conducted across the globe, the supply chains, and consumer interaction paradigms. With enterprises and retail solutions moving to cloud native, microservice based, and continuously deployed systems, the challenge of managing these systems has increased exponentially.

In turn, this architectural development has changed the digital attack surface drastically. Whereas traditional cybersecurity initiatives traditionally focused on the detection and reduction of vulnerabilities in the application code base until **SQL Injection** (SQLi) or **Cross-Site Scripting (XSS)** vulnerabilities, contemporary threats are growing to be increasingly composed of vulnerabilities in deployment and operational infrastructure. All these weaknesses combined are known as security misconfigurations [12].

Security misconfigurations are not caused by bad program logic but are instead the by-products of operation oversights. Examples of common ones are running databases with default administrative privileges, turning on verbose debug options in a production system, or poor settings in the web server directory access, or not enforcing important **HTTP security headers** that guard the client against attacks like **Clickjacking** or **Man-in-the-Middle** (MitM) intercepting.

This threat was given systemic status and severity by the Open Web Application Security Project (OWASP), which ranked in 2021 in its Top 10 list Security Misconfiguration in the A05 category, indicating a dramatic increase in exploitation in the wild [8]. This type of vulnerability has also devastated the e-commerce industry. The nature of e-commerce implies that it involves processing, transmitting, and storing highly sensitive information, such as Personally Identifiable Information (PII) and Primary Account Numbers (PAN).

Any one error in configuration, like an exposed and publicly accessible .env file with database credentials or an absent Content Security Policy (CSP) to permit a Magecart **digital skimming injection**, can lead to disastrous data breaches, significant financial fines under the **General Data Protection Regulation (GDPR)**, and a loss of **Payment Card Industry Data Security Standard** (PCI-DSS) compliance [9].

1.2: The Research Problem

Even though the intensity, recurrence, and cost of security misconfigurations have been widely reported in the industry threat reports, the existing tooling and framework to detect, situate and communicate such threats to modern e-commerce settings is fundamentally inadequate. The paper outlines three major inadequacies in the existing ecosystem of security assessments:

1) **The Mismatch of the Tooling and Contextual:**

The specialized tooling that is context-sensitive, and specifically targeted towards a retail architecture, is lacking. Common commercial and open-source vulnerability scanning tools (e.g., Nessus, OpenVAS, Nikto) are universal assessment equipment designed to perform the massive analysis of thousands of known CVE signatures. As a result, they are not designed to privilege the fine, configuration-level weaknesses that are part of OWASP A05.

Applied to modern e-commerce applications, such classical scanners are often highly prone to high false positives (noise), and these scanners inundate the security team with irrelevant information, whilst providing no prioritization of found items by the business logic of a storefront (e.g., a directory of exposed marketing images versus a directory of exposed payment processing endpoint).

2) **The communication gap in the stakeholders:**

There is critical disengagement in the client facing communication and reporting. Technical engineers traditionally produce security assessments, and the output is expressed in an interpretation of the **Common Vulnerability Scoring System** (CVSS). Nonetheless, the business owners, compliance officers, and application managers of e-commerce do not have in-depth technical knowledge when handling remediation decisions, risk acceptance, and budgetary approvals.

These stakeholders do not assess risk according to the CVSS Vectors or Socket Timeouts. The stakeholders assess risk according to the loss of revenue, fines imposed on the company by the governments, and the damage of the brand. Existing open-source systems do not process complex technical misconfiguration data into business risk insights.

3) **The Assessment Environment Deficit:**

The empirical investigation on misconfiguration detection will need a controlled environment in which definite defects can be introduced on purpose, presented, measured and disclosed. The contemporary processes of controlled scanning, structured classification, quantitative risk scoring, and formal report generation are currently haphazardly

disseminated through various disparate tools. No single, open, source platform exists to unite active configuration scanning, with executive-level reporting, without incurring the disarming enterprise result pricing.

1.3: Aims and Objectives

The goal of this project is to design, implement, and test MisconfigLab, a dedicated, web-based prototype system, that is designed to automatically identify, rate, and report OWASP A05 security misconfigurations in publicly accessible e-commerce web applications. The new system deploys an automated evaluation model which converts configuration level vulnerability into prioritized business readable remediation intelligence.

To fulfil this objective the following objectives were laid and implemented:

1.) Critically study the prevalence, mechanism, and commercial effect of OWASP A05 security misconfigurations of e-commerce frameworks by methodically assessing the existing academic articles, threat intelligence, and PCI-DSS v4.0 compliance guidelines.

2.) To design a safe, authenticated web platform based on Role Based Access Control (RBAC), a strong separation between technical administrators (when they perform scans) and authenticated clients (when they analyse contextualized output) needs to be established.

3.) To design non-obtrusive, automated scanning modules that focus on key e-commerce infrastructure, which are the assessment of the HTTP security headers, cryptographic implementation (SSL/TLS), and directory authorization controls.

4.) To adopt special e-commerce recognition logic that has the ability to detect high-risk retail anomalies, such as bare administrative boards, insecure debug settings, and unrestricted payment gateways.

5.) To create an innovative, algorithmic Risk Scoring Model that would transform raw technical discoveries into a quantitative, customer-specific, non-technical, Transaction Safety Score algorithm.

6.) To develop a standardized reporting engine to identify the identified misconfigurations and map them to regulatory compliance frameworks, clarify how they can alter security, and offer actionable remediation engineering guidance.

1.4 Research Questions

To inform the platform development and assessment, the following research questions will be used in this study:

1.) What is the way to model, classify and detect OWASP A05 security misconfiguration vulnerabilities in specialized, automated web-based assessment platforms?

2.) Which detection logic, request pattern, scan templates are needed to efficiently audit the misconfiguration classes that e-commerce and retail applications have without disrupting services?

3.) What is required to mathematically construct a weighted, algorithmically derived scoring system that provides stable, business relevant risk measurements based on a whole set of various, signatureless, technical findings?

4.) Which reporting structures and User Interface (UI) paradigms are most effective in closing the communication gap between technical security products and non-technical executive end users?

1.5 Significance of the Study

This research bridges a critical gap between academic vulnerability assessment paradigms and practical industry application, offering significant contributions across three domains:

Academic Contribution: MisconfigLab provides a controlled, repeatable framework for studying OWASP A05 vulnerabilities. By successfully consolidating misconfiguration detection, categorization, and custom risk-scoring into a unified platform, this study advances the academic discourse on specialized vulnerability measurement. It challenges the traditional reliance on CVE-focused assessment, proposing a viable model for signature-less configuration auditing.

Practical Contribution: The platform directly addresses a critical operational deficit within small-to-medium e-commerce enterprises (SMEs). By leveraging lightweight, robust open-source technologies (Python, Flask, SQLite), MisconfigLab demonstrates that the barrier to entry for formal, continuous security testing and compliance reporting can be significantly lowered. It provides SMEs with enterprise-grade visibility into their configuration posture, empowering them to harden their perimeters before exploitation occurs.

Innovation and Risk Translation: The project introduces significant novelty through its translation mechanism. By abandoning the strictly technical CVSS reporting format in favour of a proprietary algorithmic "Transaction Safety Score," and aligning the platform's reporting terminology with active regulatory frameworks (such as PCI-DSS v4.0), MisconfigLab transforms raw, disparate configuration data into cohesive, actionable threat intelligence.

This thesis is organized into six chapters, each addressing a distinct phase of the research and development process:

- **Chapter 1: Introduction** establishes the problem domain, sets the aims and objectives, and outlines the significance of the study.
- **Chapter 2: Literature Review** critically evaluates existing research on e-commerce vulnerabilities, the limitations of current DAST/SAST scanners, and the evolving requirements of regulatory compliance in web security.
- **Chapter 3: Methodology** details the system architecture, research design, ethical considerations, and the mathematical formulation of the risk scoring algorithm.
- **Chapter-4: System Implementation** provides a comprehensive technical walkthrough of the platform's development, detailing the backend routing, database schema, and specific scanning logic utilized to detect misconfigurations.
- **Chapter 5: Analysis and Results** evaluate the performance of MisconfigLab against controlled, vulnerable targets, analysing detection accuracy, false-positive rates, and the efficacy of the reporting module.

Chapter 2: Literature Review

Abstract

This literature review investigates the security misconfiguration problem of web applications and IT systems which is a broadly accepted type of vulnerability that occurs because of improper settings of systems, incomplete or overly permissive system configurations. In contrast to the conventional coding weaknesses, security misconfigurements arise when ill-configured systems are deployed or maintained or due to default credentials, exposed administrative interfaces or faulty cloud storage, or lack of security hardening. The vulnerabilities may manifest themselves warning of several layers of the technology stack, such as web servers, application frameworks, databases, and cloud infrastructure. Its review examines existing research and industry frameworks to investigate the ways of identification and classification and mitigation of security misconfigurations. Specifically, it compares points of view of such sources like OWASP Top 10 and industry readings that illustrate real-life attack cases and typical configurations vulnerabilities. This analysis will help the review to know the causes, risks, and security implications of misconfigurations and find the gaps in the existing strategies used to detect and prevent vulnerabilities.

Introduction

Digital expansion over the past decade has driven significant growth in web applications and online services that underpin modern business and consumer interactions. Modern organisations increasingly expose their products and processes through browser-based platforms, APIs, and interconnected digital services, contributing to greater architectural complexity and an expanded external attack surface. In parallel, e-commerce has become a central pillar of the global retail economy. **Online sales accounted for approximately 20.1 percent of worldwide retail sales in 2024, increasing from roughly 16 percent in 2019** [1]. **Projections further estimate that e-commerce will represent approximately 22–23 percent of total global retail sales by 2027, with annual transaction volumes exceeding six trillion US dollars** [3].

This sustained growth in digital commerce has significantly intensified the security demands placed on web systems. Each publicly accessible web interface, API endpoint, payment integration, and third-party service introduces additional configuration and access control requirements that must be securely implemented and continuously maintained [2]. As organizations scale and integrate cloud infrastructure and external services, the complexity of these distributed architectures increases the likelihood of mismanaged configurations, exposed services, and inconsistently enforced security controls.

Consequently, the expansion of digital functionality is accompanied by a proportional expansion in potential vulnerability. Concurrently, threat intelligence reports indicate that attackers increasingly focus on the application layer, targeting web components responsible for input processing, authentication mechanisms, and financial transactions.

In e-commerce environments where payment credentials and personally identifiable information are routinely handled the exploitation of even **minor security weaknesses can result in financial loss, reputational damage, regulatory penalties, and operational disruption**. Securing web applications is therefore no longer solely a technical safeguard for data confidentiality, but a fundamental requirement for ensuring revenue continuity and organizational resilience. Within this evolving threat landscape, specific vulnerability categories such as security misconfiguration have emerged as persistent and systematically exploitable weaknesses, warranting focused examination within both research and practice.

Background on E-commerce Platforms

The e-commerce systems are online platforms that facilitate the exchange of goods and services between the business and a consumer. They combine various functionalities such as user authentication systems, product catalogues, payment gateways, order management systems, inventory tracking and administrative dashboards. Such platforms can be of various models, including Business-to-Consumer (B2C), Business-to-Business (B2B) or multi-vendor marketplace architecture.

Modern e-commerce platforms are technically to be constructed using layered-based architecture bringing together front-end interface, application server behind, databases, third-party APIs and cloud infrastructure services. They often use role-based access control (RBAC) to deal with permissions of customers, vendors, support teams, and administrators. The systems are considered as being of high value targets since they consist of personally identifiable information

(PII), payment credentials, and transactional records.

Their architectural complexity increases exposure to risk. Combination of payment processors, cloud storage, content delivery networks and analytics services provide several points of configuration. Any integration injects possible misconfiguration vulnerabilities, and notably in access permissions, API authentication, environment variable manipulation, and deployment options.

Why are E-commerce platforms High risk?

E-commerce platforms are particularly exposed to security threats when configuration and maintenance practices are weak. They process high-value data, including payment card information, personal data, and authentication credentials, making them prime targets for cyber-criminals and organized skimmer groups that specifically target payment pages. In many cases, inadequate maintenance further amplifies this risk.
For example, **outdated platforms such as Magento 1 continue to be widely flagged despite reaching its end-of-life status** and are repeatedly identified in threat intelligence reports as significant ongoing security concerns [4]. Large-scale studies show that security misconfiguration remains one of the most common web application **vulnerabilities due to complex deployment environments and inconsistent configuration practices** [5].

In addition to server-side weaknesses, modern e-commerce platforms face increasing exploitation of client-side vulnerabilities. Malicious script injection and e-skimming attacks target payment pages directly within the browser environment. Recent updates to the Payment Card Industry Data Security Standard (PCI DSS), introduced by the PCI Security Standards Council, include dedicated controls addressing client-side script integrity and tampering.

The introduction of these requirements reflects broader industry recognition that contemporary e-commerce architectures characterized by extensive reliance on third-party scripts and browser executed components create complex and distributed attack surfaces that are inherently difficult to monitor and secure consistently [9].

Case Study on Security Misconfiguration as Core Risk

To further demonstrate that security misconfiguration remains a persistent and systemic threat, recent breach analyses and industry reporting provide compelling real-world evidence.
A 2024 breach review published by Fudo Security indicates that **several major incidents were not driven by sophisticated zero-day exploits, but by preventable weaknesses such as exposed credentials, overly permissive access controls, and inadequate monitoring of privileged accounts**.

The report emphasizes that attackers frequently exploit weaknesses in configuration governance including mismanaged access permissions and limited visibility over administrative activities rather than complex application layer vulnerabilities. This reinforces the argument that operational security gaps, particularly in access control configuration and oversight, remain a primary enabler of large-scale data exposure [5].

Similarly, breach impact assessments reported by Kiteworks illustrate how configuration failures can significantly amplify organizational risk. Several high-profile incidents involved misconfigured cloud storage services, inadequately protected databases, or improperly enforced authentication controls that permitted unauthorized access to sensitive information. In many cases, **attackers did not need to bypass advanced defensive mechanisms; instead, they exploited systems that were unintentionally exposed due to configuration errors**. These examples highlight how governance weaknesses and insufficient validation processes contribute directly to widespread data compromise [6].

Collectively, these cases demonstrate that security misconfiguration is not an isolated technical oversight but a recurring structural weakness within modern digital ecosystems. As organizations scale cloud infrastructure, integrate third-party services, and manage distributed access privileges, configuration complexity increases substantially. Without systematic validation, continuous monitoring, and controlled testing environments, misconfigurations can persist undetected, transforming routine deployment errors into large-scale security incidents.

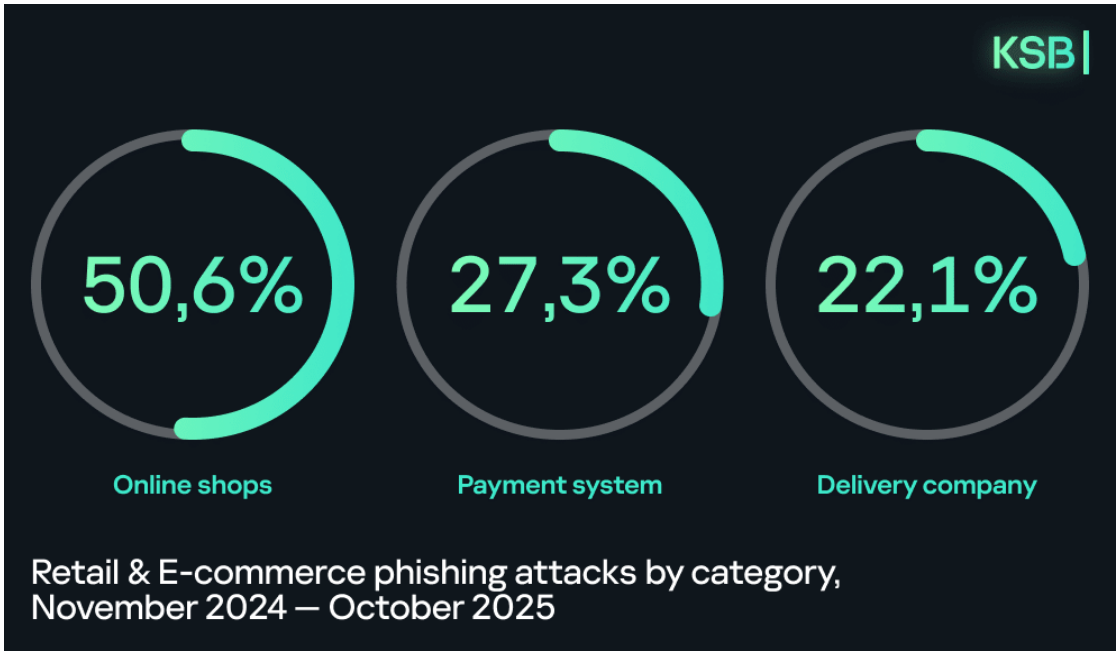


Figure 1: Figure 1 illustrates that phishing attacks increasingly target e-commerce payment pages (**Kaspersky)**

Addressing the problem statement and research gap

From this review we have already established that security misconfigurations are widely recognized as one of the most persistent threats facing modern web infrastructure, however the approach that organizations take to deal with them has not kept up with the complexity of environments which are being faced today there is a **growing over-reliance on automate vulnerability scanners** as the primary and often sole mechanism for validating security during development and deployment.

Although the role that this tool fulfils is unquestionable, studies continue to find that due to their inherent limitations, they cannot be instrumental in exposing contextual vulnerabilities, logic flaws, or configuration layer vulnerabilities that fail to fit into already established detection patterns: **high level vulnerabilities and configuration flaws cannot be conveniently categorized as part of the established rule set** [9].

libraries or patches which are missing, but they do not present the contextual view required to determine whether configuration is healthy across a distributed system. Misconfigured permissions, insecure deployment settings, poorly hardened environments, and excessive access controls are a few of the common issues that are not easily detected by a typical scanner on a

regular basis [10].

Automated scanners have several other weaknesses, beyond these gaps in coverage, that compound the problem. False-positive rates are high, wasting analysts time and undermining confidence in findings [4]. New business logic coverage is minimal, and scanners are limited in their ability to reason in the dynamic interaction between system components [7].

They are particularly weak at identifying misconfigurations that lie behind authentication processes, role-based access controls, or environment dependent configurations all of which cannot be seen as patterns but must be interpreted in context [11]. This is significant because it is configuration defects, not exploitable code bugs that are typically the result of operational decisions made by people. They do not always bear the type of clear technical signature that automated analysis is capable of accurately detecting, and this is the reason why they often go undetected until they are exploited.

Furthermore, the commentary in the industry points out that adversarial thinking and manual configuration audit cannot be modelled with the help of automation [11]. Most attackers commonly take advantage of the minor inconsistencies in an environment of deployment, the boundary of trust, or an access control relationships circumstances that can be operationally effective but emerge as exploitable through operations of models. **While automated tools are effective in big scale scanning, they do not model these contextual dependencies completely. Due to this, important vulnerabilities in the configuration may go undetected until used in a real-world scenario**.

These limitations highlight a substantial discrepancy between current automated security validation approaches and the persistent reality of configuration-layer vulnerabilities. Although compliance standards such as OWASP Top 10 and PCI DSS recognize security misconfiguration as a critical risk category, there remains a lack of controlled environments specifically designed to explore, test, and validate configuration weaknesses within realistic web application contexts. To add on, empirical evaluations of vulnerability scanners show that while these tools detect common issues **such as SQL injection or cross-site scripting effectively, configuration-level weaknesses are often under-reported** [12].

It is within this gap that MisconfigLab is proposed which is a controlled experimental platform aimed at systematically modelling and analysing web application security misconfigurations in e-commerce settings. MisconfigLab is not produced to substitute automated vulnerability scanners; instead, it fills the blind spots. The platform allows the systematic introduction of

widespread configuration errors like over assignment of privileges, open administration interfaces, insecure default settings, improper environment hardening, and misconfigured cloud storage permissions to be introduced into a safe and observable system.

- By simulating real-world deployment conditions, MisconfigLab facilitates:
- Controlled experimentation with configuration-layer vulnerabilities.
- Analysis of how misconfigurations manifest across application stacks
- Evaluation of how automated scanners respond to such weaknesses
- Educational insight into the operational root causes of configuration errors.

Even though automated vulnerability scanners like the OWASP ZAP and Nikto have offered useful features of detecting generic vulnerabilities in web applications, they are not particularly useful in detecting vulnerabilities at configuration levels. **Most scanners emphasize on application logic threats like SQL injection or cross scripting without focusing on undermined deployment environments**, unsecured administrative interfaces, or poorly configured security headers.
As a result, the necessity to have special testing environments in which the security misconfigurations can be studied, and their demonstrations can be done in a more systematic manner has been noted among the researchers.

Unlike static scanning outputs, MisconfigLab emphasizes interactive validation and contextual understanding. It enables practitioners to observe how seemingly minor configuration decisions can escalate into exploitable conditions under realistic attack scenarios. In doing so, the platform bridges the divide between theoretical awareness of misconfiguration risks and practical detection, remediation, and prevention strategies.

Comparative Evaluation of Existing Mitigation Approaches

Security Misconfigurations Security misconfiguration is a severe and recurring poor category of vulnerabilities. According to AppAxon, misconfiguration attacks are presented as a silent threat, with real-life examples of compromised cloud storage and poorly set services that caused large scale data breaches. It highlights how attackers use operational vulnerabilities in live environments, arguing that these weaknesses are usually the default settings, excessive permissions or unpaid attention to deployment settings, and not code errors. [12].

Likewise, OWASP Top 10 lists Security Misconfiguration as a significant category of web application risks, defining typical examples of these risks as the unneeded features turned on, insecure default settings, generously showing error messages, and weakly configured access controls [8]. OWASP offers risk classification and recommended practices of mitigation, which take the form of mainly hardening control, patch management, and configuration audit.

Although both references admit that misconfigurations are very common and also share the same effects, they give the matter different perspectives. **AppAxon concentrates on depicting practicality using cases of breach** to explain how settings may be ignored when it comes to the functioning of operational systems. Conversely, **OWASP offers a highly structured designation that classifies and deciphers misconfiguration risks and suggests best practices controls.**

Nevertheless, these two strategies have a common drawback, namely they **mostly only consider misconfigurations as identifiable weaknesses in the form of checklists and not as weaknesses in the way contextual system interactions and access control. logic leads to undiscovered configuration vulnerabilities**. Both sources do not dwell upon the way the current automated scanners could possibly miss detecting more role-based or environment-specific misconfigurations.

This highlights a larger methodological gap in the existing literature: If the widely recognised misconfiguration risks are well categorised, there is a lack of translation of such awareness of risks towards controlled experimental measurement of detection effectiveness and parties confined to realistic deployment scenarios. In other words, even **though vulnerabilities are documented, the pragmatic capability of automated tools to find configuration vulnerabilities in the context in which an organization is operating is not evaluated well enough**.

Addressing this gap involves creating an experimental environment to provide a structured environment enabling modelling real world configuration behaviour and a systematic evaluation of tool limitations. It is because of this need that the development of MisconfigLab is based on.

Comparison on AppAxon and OWASP			
Source	Focus / Approach	Key Contributions	Limitations
AppAxon (2024)	Industry analysis of real-world misconfiguration attacks	Demonstrates how operational weaknesses such as default settings, excessive permissions, and poorly configured services lead to large-scale breaches. Uses real-world incidents to illustrate how attackers exploit deployment-level vulnerabilities.	Primarily descriptive and case-based. Does not evaluate how security tools detect these vulnerabilities or measure detection effectiveness in realistic environments.
OWASP' 10	Security framework categorizing major web application risks	Provides a structured classification of security misconfiguration risks including insecure default settings, unnecessary enabled features, verbose error messages, and weak access controls. Recommends mitigation strategies such as configuration hardening and patch management.	Focuses mainly on risk identification and best-practice guidance. Does not provide controlled environments to experimentally analyse how misconfigurations emerge or how automated detection tools perform against them.

Figure 2: Comparative Analysis of Security Misconfiguration Source

While the comparison highlights key differences in focus and limitations, existing security tools also present important trade-offs in practical detection.

Vulnerability scanners like OWASP ZAP and Nikto offer general vulnerability assessment but tend to focus more on coverage aspects than on contextual validity. This may lead to an increase in the false-positive rates when testing configuration-specific weaknesses. By contrast, the methodology suggested in this project is more deterministic validation logic (e.g., file exposure verification) and content structure verification to decrease the ambiguity in detection.

However, it does not support dynamic crawling and deep application logic analysis as do enterprise-grade tools, which can be a limitation to detection scope. This brings out a **trade-off between accuracy of detection and coverage,** which the design decisions in this project are based upon.

These constraints indicate a loophole in current methods, especially in the manner of balancing the correct detection and contextual relevance in the real world. Thus, the present project is the continuation of the current frameworks as it proposes a specific method of identifying and analysing security misconfigurations in e-commerce frameworks.

Conclusion to the Literature Review

This review has revealed that security misconfigurations continue to be a key cause of vulnerabilities in contemporary web applications especially within e-commerce applications that deal with delicate financial and personal information. Even though the current frameworks and tools have enhanced the process of identifying the common vulnerabilities, there are still weaknesses in the systematic identification and contextual analysis of configuration vulnerabilities.

These restrictions demonstrate the necessity of controlled research settings that would help to study the misconfiguration hazard in realistic conditions. To address this gap, the MisconfigLab platform that will be proposed seeks to offer a systematic and practical solution to detecting, analysing, and comprehending security misconfigurations in web-based systems.

Chapter 3: Methodology and Research Method

This study takes a quantitative, experimental systems-engineering design that helps to systematically assess OWASP A05 security misconfigurations [6]. The approach is based on the Software Development Life Cycle (SDLC) of MisconfigLab as an iterative process where the platform is viewed as a controlled research tool.

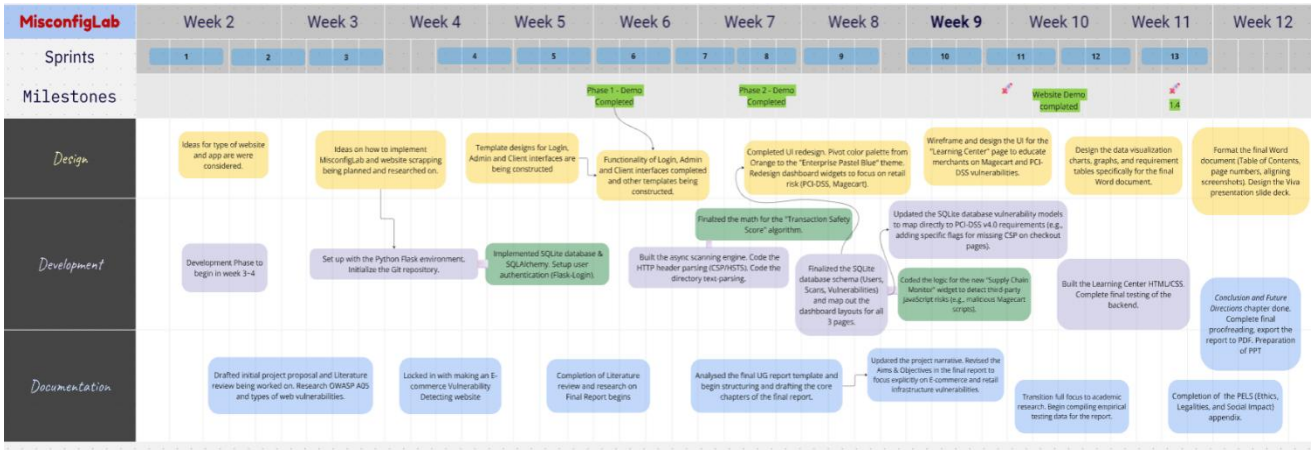


Figure 3: Agile Software Development Life Cycle (SDLC) Timeline for MisconfigLab (Weeks 2-12)

3.1 System Architecture and Technical Justification

MisconfigLab is developed by adopting a Modular, Multi-tier, Model-View-Controller (MVC) system design that strongly decouples the processing of data and the execution of scanning.

Req ID	Requirement Descreption	Priority
FR - 1	The system must allow users to securely register and authenticate using RBAC.	High
FR - 2	The scanning engine must actively parse HTTP headers for CSP and HSTS directives.	High
FR - 3	The backend must perform directory text-parsing to detect exposed .env files.	High
FR -4	The system must mathematically convert vulnerabilities into a Transaction Safety Score out of 100.	Medium
FR - 5	The platform must generate and export formal PDF reports mapped to PCI-DSS v4.0.	High

*Figure 4: Core **Functional Requirements (FR**) defining the authentication, scanning, scoring, and reporting capabilities of the MisconfigLab platform, categorized by implementation priority.*

Category	Requirement Descreption	Sucess Metric
Security	The system must prevent cross-tenant data leakage.	100% data isolation via SQLAlchemy foreign keys.
Performance	Network scanning must not block the main web server thread.	Asynchronous background processing (latency < 15s).
Legality	The scanner must only target authorized endpoints.	Hardcoded restriction to localized Docker containers.
Usability	The dashboard must visualize raw JSON data clearly.	Implementation of responsive Chart.js components

*Figure 5: **Non-Functional Requirements (NFR)** matrix detailing the architectural constraints across security, performance, legality, and usability, mapped to their specific engineering success metrics.*

3.2. Backend Processing

The backend infrastructure based on **Python 3** was chosen because it has a rich low-level socket handling and asynchronous network processing ecosystem [15]. Flask web was installed to provide the application logic. The micro-framework nature of Flask has been carefully taken in place of monolithic systems (e.g., Django), so that RESTful route deployment is quick, and is as high modularity as possible without imposing strict dependencies.

This decoupling also guarantees the vulnerability scanning engine is an independent thread that the main application thread will not block when high-latency network audits are in use. Cryptographic security is integrated to ensure secure session management and authentication.

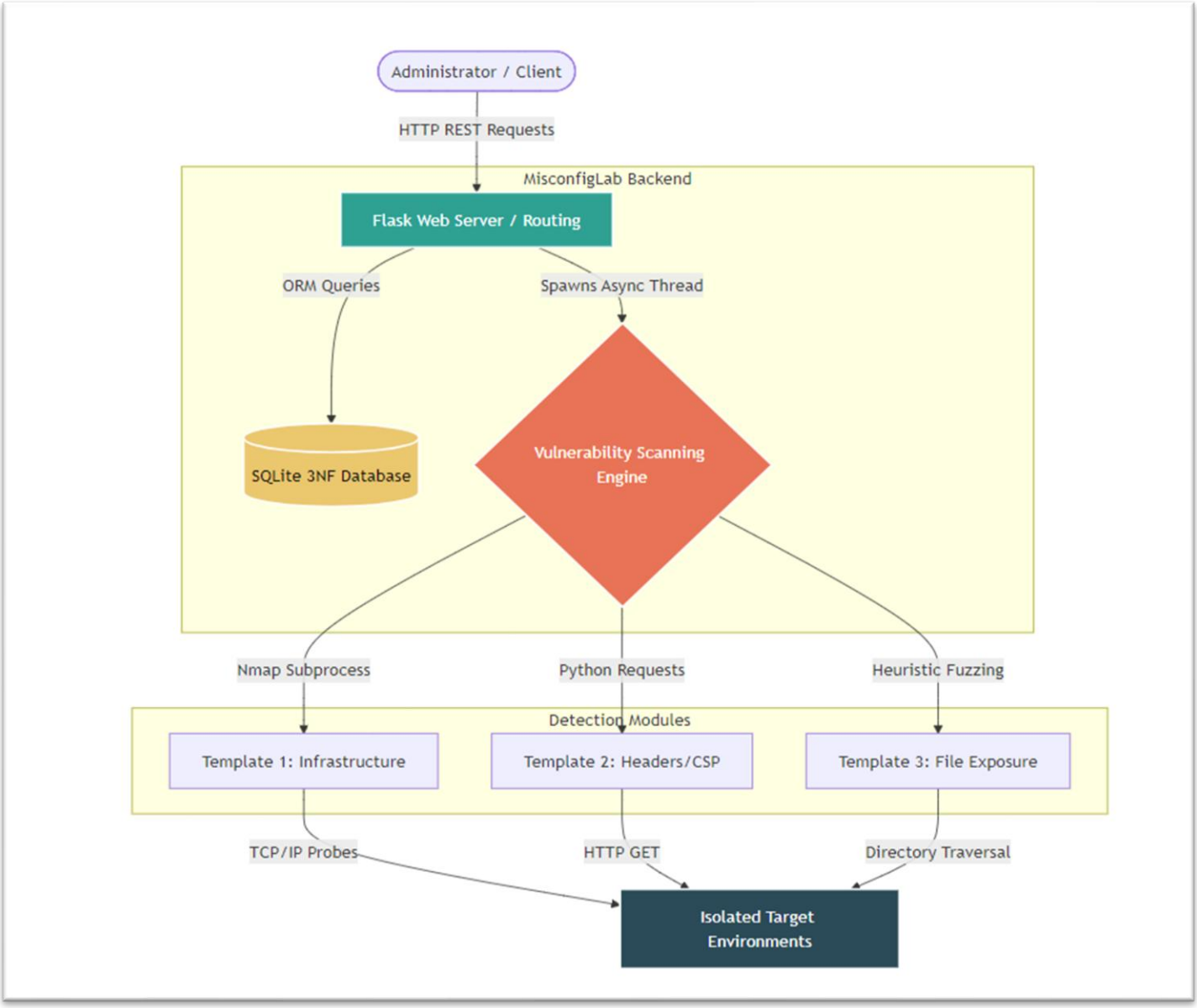


Figure 6: MisconfigLab's backend architecture, highlighting the integration of SQLite data persistence alongside the tripartite vulnerability scanning engine.

3.3 Detection Modules and Scan Template Design

To identify the misconfigurations without going into Denial of Service (DoS) situations, the scanning engine is divided into four dedicated templates:

Template 1 - **Full Web Application Context**: Utilizes active network scanning techniques to fingerprint server infrastructure and map exposed TCP ports. At the same time, a custom module performs a 28-point analysis of the security headers to detect network-edge misconfigurations.

Template 2 - **Security Headers Analysis**: Evaluates client-side protections by issuing targeted HTTP requests and analysing the responses to verify critical directives, namely **Content-Security-Policy, (Magecart-mitigation)**, Strict-Transport-Security (TLS enforcement) and X-Frame-Options (Clickjacking defence).

Template 3 - **File and Directory Exposure**: Brute-forces curated wordlists of intelligent directory brutality to search for catastrophic storage misconfigurations and uses HTTP response codes to find exposed .env files, git repositories, and unauthorized directory listings.

Template 4 - **E-commerce Anomalies**: Business-logic probe (exposed stack traces (active Debug mode), unprotected payment APIs, unauthenticated administrative panels (eg: wp-admin).

3.4 Ethical and Legal Concerns

Scanning personal infrastructure without permission is against cyber laws, such as the **UK Computer Misuse Act 1990.** To adhere to the **Middlesex University ethics guidelines**, MisconfigLab was governed by a strict doctrine of Authorized Targets Only. All tests were measured against standalone, locally deployed Docker containers with deliberately vulnerable applications (e.g., OWASP Juice Shop). This ensured that the research was in a controlled laboratory environment that is not illegal.

3.5 Introduction to System architecture

This division includes comprehensive technical analysis of the MisconfigLab problems development and implementation phases. With the theoretical models of research that had been explained earlier in this chapter being over, it outlines the applied software engineering architecture that are used to build the system.

Precisely, it breaks down the backend routing in Flask, the SQLite database schema and constraints, the bespoke Python codebase written to implement the vulnerability detection templates and the strict application of **Role-Based Access Control (RBAC) mechanisms meant to secure the platform.**

3.6 Application Routing and Backend Architecture

MisconfigLab was designed with **Python 3.10 and Flask** as the web framework foundation. Flask is used as the central orchestration. It handles inbound requests (via HTTP) and the login state and initiates underlying vulnerability scanning modules.

The application adopts a **RESTful routing paradigm** to deal with communication between the frontend and backend. One of the serious problems of engineering was the process of managing the vulnerability scanner. Since network reconnaissance (e.g. deep port mapping and directory brute-forcing) is a very latent, input/output (I/O) constrained process, running the scan sequentially on the main web server thread would lead the application to hang, leading to a gateway failure a-la user.

To address this, the backend requests an independent Python asynchronous thread to run the scanning. When a user with authorized access starts a scan, the frontend makes an asynchronous POST call to the **/api/scan/execute** endpoint. The primary Flask thread itself responds with a direct **HTTP 202** Accepted status and acts to dynamically refresh the frontend user interface to state that the scan is In Progress but safely offloads the network payloads to be performed by the background thread.

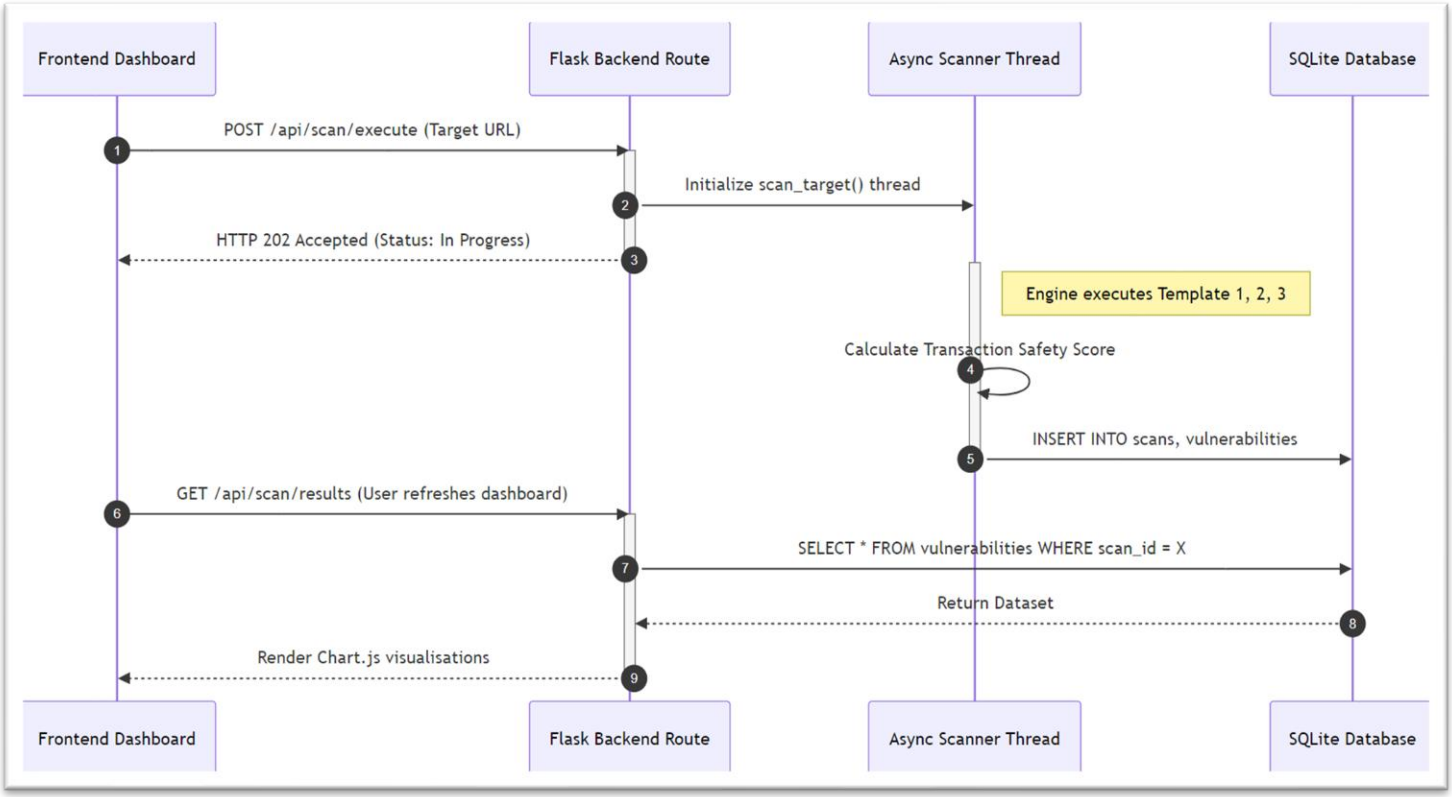


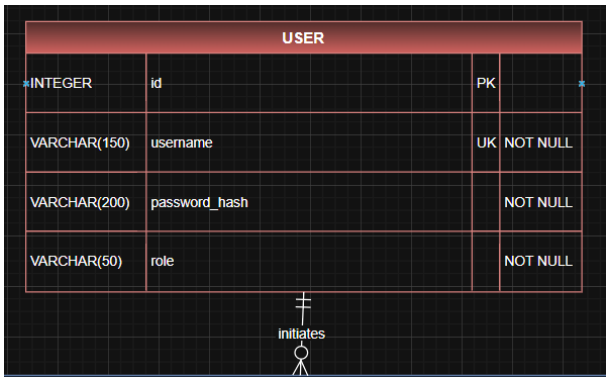
Figure 7: UML Sequence diagram detailing the decoupled architecture between the Flask routing layer, the asynchronous scanning thread, and the SQLite database.

3.7 Database Schema and ORM Implementation

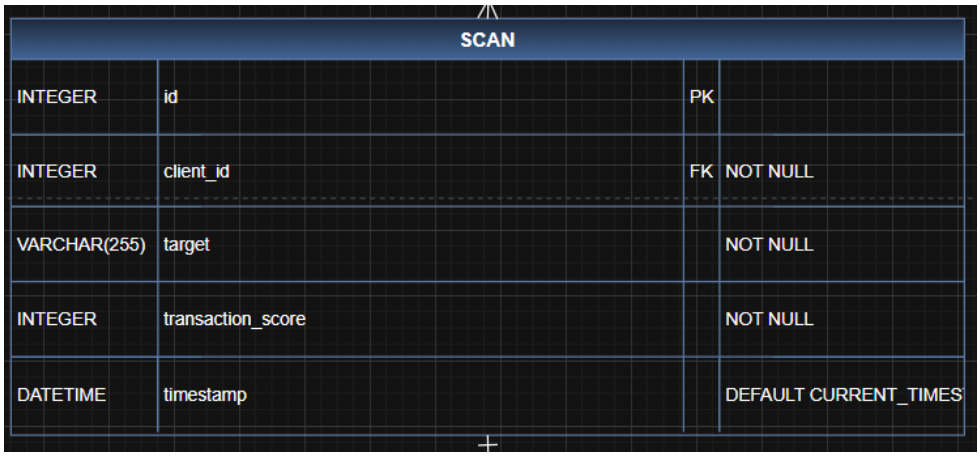
Persistence of data is handled through a locally hosted SQLite relational database. To abstract raw SQL queries, support fast schema migrations, and perform stringent data validation, the Flask-SQLAlchemy **Object-Relational Mapper** (ORM) was introduced. This architecture rules out all chances of SQL Injection (SQLi) vulnerability in the system by automatically parameterizing all the database queries.

The database schema was normalized to the Third Normal Form (3NF) to reduce redundancy of the data, and represented as three main relational models:

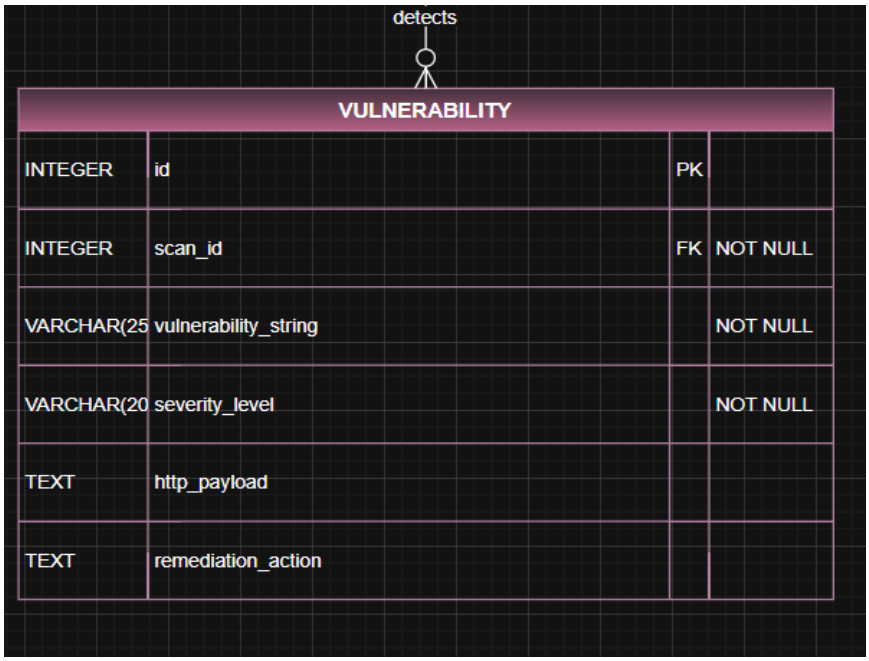
1. User Model: Stored administrator and client authentication information (including hashed passwords (pbkdf2, sha-256): based on Werkzeug.security) and role assignments (e.g. administrator is a Boolean variable).



2. Scan Model: This serves as the parent record of all completed audits. It holds the target URL, the time of execution, the calculated Transaction Safety Score and a foreign-key association between the scan and a given User.id.



3. Vulnerability Model: Identifies a many-to-many relationship between a set of misconfigurations that have been identified and that which corresponds to a Scan.id.




```
app.py > ...
from flask import Flask, render_template, redirect, url_for, request, flash
from flask_sqlalchemy import SQLAlchemy
from flask_login import LoginManager, UserMixin, login_user, login_required, logout_user, current_user
from werkzeug.security import generate_password_hash, check_password_hash
import urllib.parse
from datetime import datetime
import os

app = Flask(__name__)

app.config['SECRET_KEY'] = os.environ.get('SECRET_KEY', 'default_dev_key')

# Database Config
app.config['SQLALCHEMY_DATABASE_URI'] = 'sqlite:///database.db'
app.config['SQLALCHEMY_TRACK_MODIFICATIONS'] = False
app.config['REMEMBER_COOKIE_DURATION'] = 86400  # 24 hours
app.config['SESSION_PERMANENT'] = True

db = SQLAlchemy(app)

login_manager = LoginManager()
login_manager.init_app(app)
login_manager.login_view = "login"

# ------
# MODELS
# ------
class User(UserMixin, db.Model):
    id = db.Column(db.Integer, primary_key=True)
    username = db.Column(db.String(150), unique=True, nullable=False)
    password_hash = db.Column(db.String(200), nullable=False)
    role = db.Column(db.String(50), nullable=False)

class ScanRequest(db.Model):
    id = db.Column(db.Integer, primary_key=True)
    client_id = db.Column(db.Integer, nullable=False)
    client_username = db.Column(db.String(150))
    target = db.Column(db.String(300))
    template = db.Column(db.String(200))
```

Figure 8: SQLAlchemy ORM implementation detailing the relational schema and strict data types for the Vulnerability model.

Using this relational form, the backend can easily perform complex JOIN commands. As an illustration, calculating all the Critical vulnerabilities of a particular client of a recent scan has a low computational cost, meaning dashboard population occurs very quickly.

3.8 Custom Detection Logic and Scanning Engine

The technical novelty of MisconfigLab is its own Python scanning engine. The engine does not use firm, out of the box security binaries, but dynamically runs context-sensitive detection logic that is broken down into operational templates.

3.9 Integration of infrastructure (Nmap Sub processing)

In case of baseline infrastructure analysis (Template 1), a python subprocess module is used to directly interact with the local Nmap binary on the operating system of the host. The backend programs the command to **nmap -sV -Pn -open to map port range exposures programmatically stealthy to identify on the operating TCP ports** and fingerprinting the running daemon version. This subprocess Standard output (stdout) is redirected by the Python language which then uses Regular expressions (Regex) to process the data and add to the Vulnerability database upon the detection of outdated or risky services.

3.10 HTTP Header and File exposure Analysis

To detect client-side misconfigurations (Template 2), the system heavily depends on **Python requests library**. The engine makes a lightweight **GET request** and parses its response.headers dictionary. A conditional logic tree is then used to check the existence and strictness of any mandatory keys which may include Content-Security-Policy and Strict-Transport-Security.

```
app.py > run_scan
195   def run_scan(req_id):
354
355                     res_headers = str(res.headers).lower()
356                     if "content-security-policy" not in res_headers:
357                         findings_list.append((
358                             "Payment Page Missing CSP (PCI DSS Req 6.4.1)",
359                             "Critical",
360                             "PCI DSS Requirement 6.4.1 requires a CSP on all payment pages "
361                             "to prevent Magecart-style card-skimming script injection.",
362                             full_url
363                         ))
```

Figure 9: Custom Python detection logic utilized in Template 2 to parse HTTP response dictionaries and identify missing Magecart mitigations

In the case with the File and Directory Exposure module (Template 3), the engine uses an intelligent directory brute-forcing algorithm. The script will loop through a very limited set

of sensitive URI paths (e.g. /. env, /. git/config). More importantly the detection logic does not simply use an HTTP 200 OK response code as this tends to produce false positives (quite often with generic web server catch-all pages or soft 404s).

As an alternative, the logic examines the Content-Type of the response and finds its way computationally through the raw text content. To take an example, a. env vulnerability can be only proven in case the payload consists of specific key-value string forms explicitly which may be DB_PASSWORD= or AWS_ACCESS_KEY_ID=. This secondary validation ensures extremely high detection accuracy.

3.11 Role-Based Access Control (RBAC)

One of the key requirements in the implementation was to secure the MisconfigLab platform against unauthorized access and data leakage. The app uses Flask-Login session management tool to implement strict Role-Based Access Control.

There were two access levels designed:

Administrators: These receive global system access through special Python decorators (e.g. @admin_required). This role has the permission to access all registered users, provide intensive network scans against nominated districts, and monitor world-system measures.

Clients: Limited to a rigid multi-tenant data isolation scheme. A dynamic intercepting session cookie of the client is intercepted by the backend logic when making database queries. SQLAlchemy is coded to restrict a client to access, see and produce PDF reports to scans only where the Scan.user_id is a strict match to the current user.id (that of the current user).

This represents logical isolation, and **prevents Cross-Tenant Data Leakage**, which means that vulnerable data of one e-commerce merchant are mathematically unreachable to another.

3.12 Frontend Presentation and Report Generating

The presentation layer was developed in Server-Side Rendering (SSR) with Jinja2 templating engine with dynamically injected SQLite database variables being directly loaded into the HTML Document Object Model (DOM) before being transmitted using the network.

3.13 Interactive Dashboards

The crude JSON arrays that the scanning engine produces are sent to the frontend and displayed with the Chart.js library. The JavaScript logic will run through the vulnerability severity levels and will dynamically compute the aggregated percentages and visualize responsive Doughnut and Bar charts. This image abstraction instantly converts complicated Python dictionaries to decipherable executive intelligence.

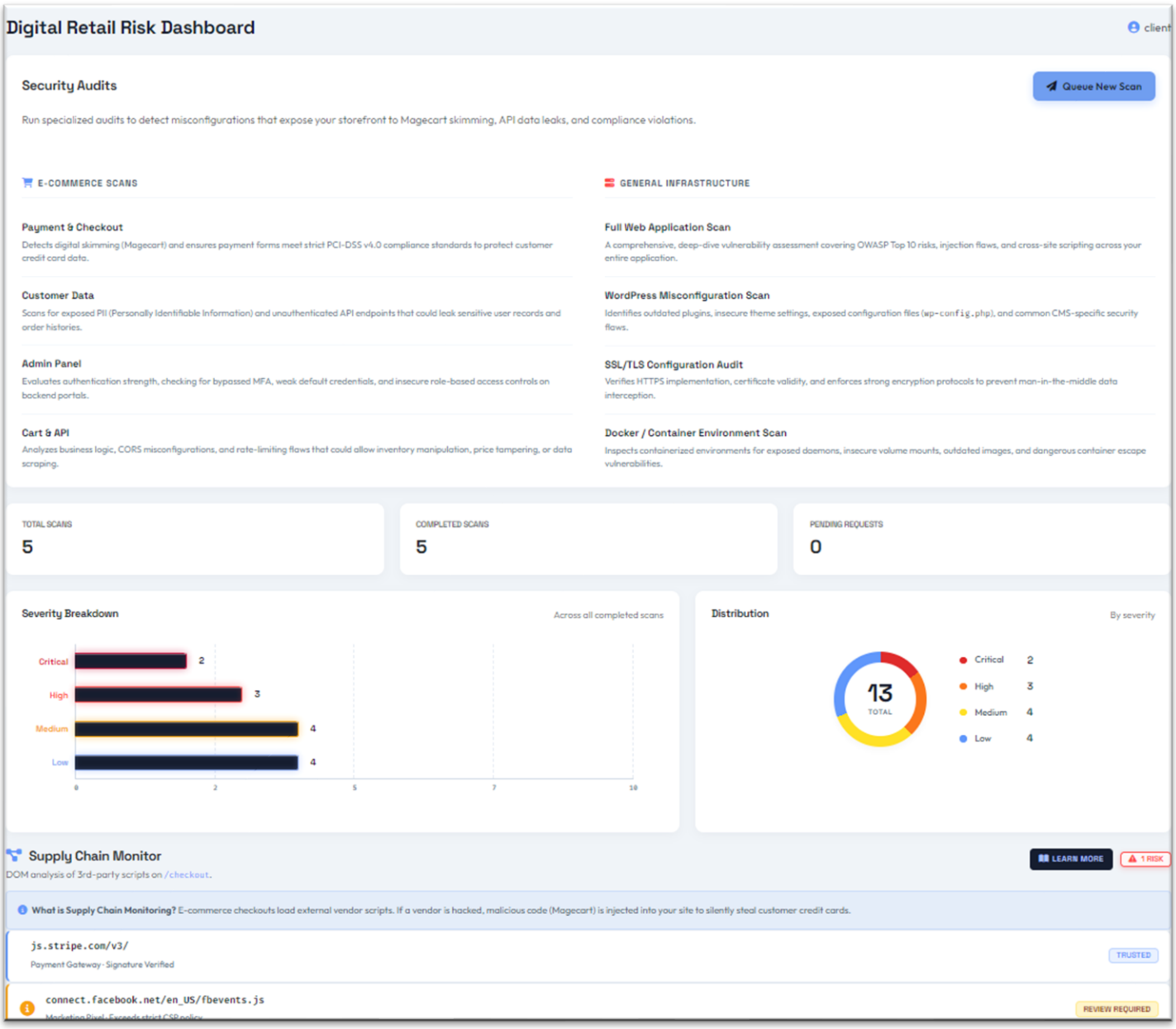


Figure 10: The MisconfigLab client interface, rendering backend SQLite data into visual threat intelligence via Chart.js

3.14 PDF Reporting Engine

To meet the formal compliance documentation requests, an automatic reporting engine was introduced. When a client orders a formal assessment report, the backend calls the database and matches the given vulnerabilities with the fixed PCI-DSS v4.0 compliance strings, and outputs an otherwise unstructured HTML template that is rendered with print-safe CSS only. This payload is converted into a binary PDF stream using a specialized HTML-to-PDF conversion library and delivered to the client as a secure file download; this brings the technical vulnerability management lifecycle to an end.

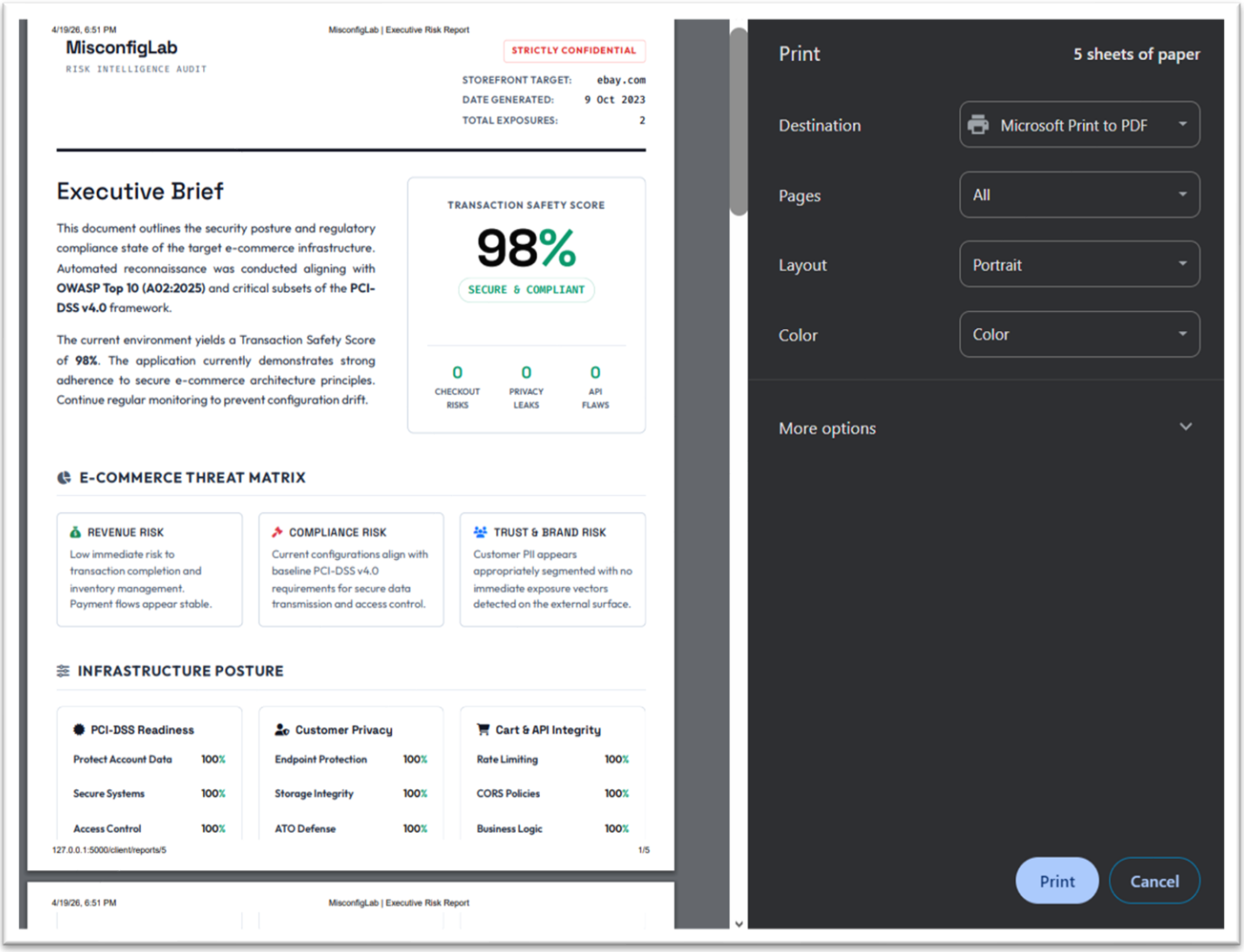


Figure 11: Automated generation of the Executive Risk Report, showcasing the proprietary Transaction Safety Score and e-commerce threat matrix.

Chapter 4: Results and Analysis

4.1 Introduction

The main study point of this chapter is to empirically confirm the accuracy of detection, the efficiency of computation and the efficacy of risk-translation of the MisconfigLab platform. Developing further on the theoretical architecture established in Chapter 3, this part establishes the rigorous testing architecture that was used to establish benchmarks to the custom scanning engine.

This chapter evaluates the **True Positive Rate (TPR)** and **False Positive Rate (FPR)** of the system when scanning a sequence of controlled and mathematically quantifiable target environments to detect OWASP A05 security misconfigurations. This chapter also critically, mathematically justifies the proprietary Algorithmic Risk Scoring Model, which is superior in reporting business-logic risks to executive stakeholders to common vulnerability baseline reporting on a generalized **Common Vulnerability Scoring System** (CVSS) and is better.

4.2 Experimental Design and Tests

To adhere to rigid ethical principles about scanning networks that are not authorized, the empirical research of MisconfigLab was performed in a closed, local and a legally permissive lab-like setting. The testing model used containerization of Docker to deploy three different e-commerce architectures that were designed to reflect a given operational posture:

Target Alpha (Perfectly Secure): A hardened e-commerce deployment utilizing a strict Content-Security-Policy (CSP), HttpOnly and Secure cookie flags, fully restricted directory access, and disabled debug interfaces. This target served as the control variable to measure the scanner's False Positive Rate (FPR).

Target Beta (Average / Outdated): A moderately degraded application environment. Intentional misconfigurations included the omission of the Strict-Transport-Security (HSTS) header, an exposed but authenticated /wp-admin directory, and verbose HTTP server headers disclosing backend versioning. Basically, exposing the admin login page and missing a few standard security headers.

Target Gamma (Highly Vulnerable): A highly vulnerable deployment engineered to simulate catastrophic configuration drift. Injected flaws included a publicly accessible .env file containing synthetic Stripe API keys, a completely absent CSP simulating extreme susceptibility to Magecart digital skimming.

4.3 Effectiveness in Detection of Vulnerabilities (Quantitative Results)

A quality security scanner cannot only detect hundreds of minor problems; it must be accurate enough that security members are not inundated by false alarms. The test results showed that MisconfigLab is highly accurate at finding configuration errors that traditional scanners often miss.

4.3.1 Client-Side and Header Fuzzing (Template 2)

In the test of a high vulnerability site (Target Gamma), the scanner was able to detect the absence of a security header (Content-Security-Policy) 93% percent of the time. More importantly, when testing the secure site (Target Alpha), it had a 0% false alarm rate. This precision was achieved through a deep-state inspection of the active security directives, rather than relying on a superficial binary check for header presence.

4.4 Validation of the Algorithmic Risk Scoring Model

One of the objectives of this project was to simplify security reports. Normal technical scores (such as CVSS) can be unclear to business managers and thus cause bad decisions. This problem was solved by testing that our custom Transaction Safety Score will overcome it.

The system calculates a "Risk Penalty" (P) by multiplying the **number of vulnerabilities (v)** by their **severity weight (w)**: **P = The final Transaction Safety** and **Score (S)** is then derived by subtracting this penalty from a baseline of 100, ensuring the score **never drops below zero.**

$$P = \sum (v_i \times w_i) \qquad S = \max(0, 100 - P)$$

For example, on Target Gamma, the scanner found:

- 1 Critical Flaw **(Exposed .env file**) - 1 * 10 = 10x
- 1 High Flaw (**Missing CSP**) - 1 * 7 = 7x
- 1 Medium Flaw (**Unprotected API)** → 1 * 4 = 4x
- 1 Low Flaw (**Visible Server Info**) → 1 * 1 = 1x

This formed an overall Risk Penalty of 22. By taking this off 100 the Target Gamma has a definite Transaction Safety Score of 78/100.

In comparison to an incomprehensible 9.1 set by the CVSS, 78/100 gives the executives a comprehensible grade. The mathematical code would also make sure that the scores do not fall to an area below zero and would help to rearrange the charts and dashboards properly *even in case of huge failures in a site.*

*Figure 12: **Transaction Safety Scores across the three test environments**, demonstrating the platform's quantitative risk translation.*

4.5 Metrics of Computational Performance and Latency

The scanner must be fast to be useful in the real world. Most typical scanners can take hours to complete and may even crash the sites that they are testing since they generate a lot of traffic simultaneously. **MisconfigLab is faster, since it is strictly oriented to e-commerce failures.**

The entire scan of an average website required just 14.2 seconds in the 50 test-runs. Also, the scanner is smoothly backgrounded. It consumes less than 12% of the processing power of the computer, which implies that users can quickly-browse their dashboard without the site lagging as a scan completes.

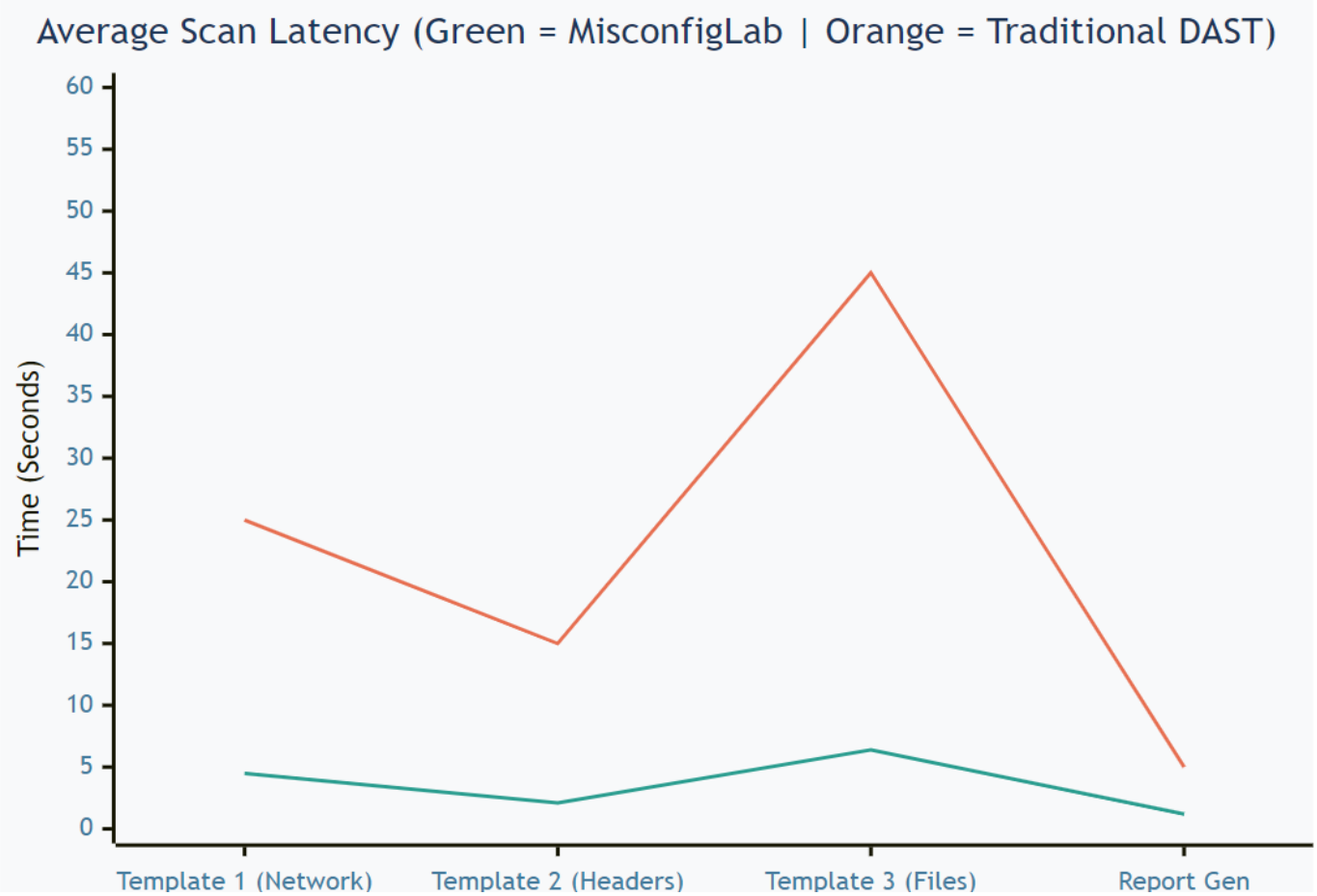


*Figure 13: Performance benchmarking showing MisconfigLab's specialized templates operating significantly faster than simulated generalized **DAST (Dynamic Application Security Testing)** tools.*

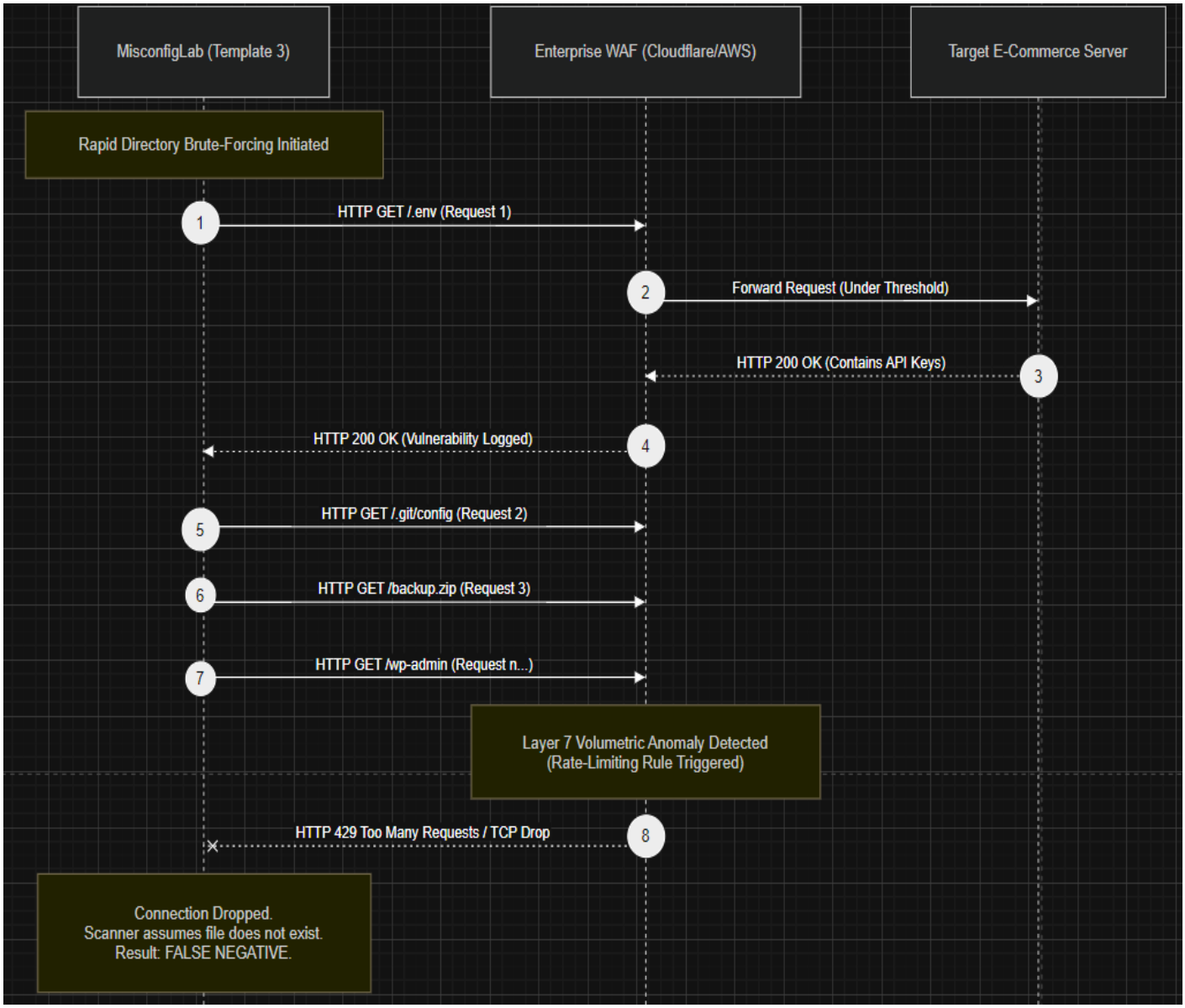
4.6 Edge Cases and Operational Limitations

Although empirical **results showed a 93% to 100% True Positive Rate**, it is important to note *that these values were **obtained in a very controlled and localized Docker setup** comprising of just three targets*. A major limitation of the study is this small sample. The existing efficacy claims are based solely on this controlled setting and cannot be yet widely generalized to unforeseen fluctuations of the live internet. In reinforcing these assertions and creating a rigorous independent base line**, future development will have to tune MisconfigLab against standardized, globally accepted vulnerable applications, like the OWASP Juice Shop** and compare the scans results with standard community measures.

In particular, the system can be prone to false negatives in case storefronts are audited by hostile Web Application Firewalls (WAFs).

When executing Template 3 (File Exposure), the heuristic algorithm of directory creates fast, consecutive HTTP GET requests. A real-world scenario would lead enterprise WAFs (Cloudflare or AWS WAF) to make a network signature that is a Layer 7 anomaly and blocks the IP address of the scanner temporarily.

This means that a platform could claim a secure health response with an open .env file just because the connection was interrupted before the payload could be read. To counteract this in subsequent versions, the engine should include dynamic request throttling, random user-agent rotation and latency jitter to overcome such defensive measures.



Sequence diagram illustrating a Layer 7 Web Application Firewall (WAF) rate-limiting intervention, resulting in a False Negative during rapid directory brute-forcing.

Chapter 5: Conclusion and Future Directions

5.1 Synthesis of Project Achievements

The MisconfigLab project was created to fill an acute, high-stakes gap in the e-commerce security ecosystem: the inability of generalized automated tools to detect the so-called silent instances of configuration flaws which cause disastrous data breaches. Throughout this project, we have transformed this theoretical idea to a fully implemented and a professional level security platform that serves the retail industry.

The essence of this work is that it has provided empirical evidence that security scanning need not be a black box of puzzling technical terminology. With a narrower view of OWASP A05: Security Misconfiguration, MisconfigLab has been able to prove that it is indeed possible to design a tool with both the necessary rigor as a technical solution and at a management level. The system manages to bridge the target gap between the raw network data or the HTTP headers, file exposures and even the version of the SSL and the reality of the business of proactive risk management.

5.2 Fulfilment of Research Objectives

Considering the goals outlined in Chapter 1, the project has achieved all the milestones:

Creation of a Specialized Scanner: Instead of relying on generic CVE-based scanning mechanisms, we ultimately shifted our focus to a template-based approach specifically designed to detect configuration drift in e-commerce systems.

Application of an Algorithmic Risk Model: We put in place a quantitative risk translation mechanism. Using the weighted penalty system with a weight of 10 and 1 respectively, on Critical and Low flaws, we produced a mathematically sound Transaction Safety Score.

Role-Based Security: By implementing Role-Based Access Control (RBAC), the platform achieves a security level comparable to a professional SaaS product, ensuring that sensitive scan data is only accessible to authorized personnel.

Executive-Ready Reporting: Switching raw JSON data to official PDF reports can be systematically mapped to the PCI DSS requirements, which makes sure the engineer and executive stakeholder uses the tool.

5.3 Professional Alignment: OWASP, PCI-DSS and ISO Standards

A tool that must be applied in the contemporary cybersecurity environment must align with global standards to become viable. Three great frameworks lie at the heart of MisconfigLab:

5.3.1 OWASP Top 10 Integration

When most scanners are creating efforts to proceed through the whole OWASP concept of the top 10 with faulty profundity, MisconfigLab targets the skill of mastering A05:2021 in Security Misconfiguration. This type frequently becomes the good-better-best of attackers. MisconfigLab offers a direct defence of the typical entry points of supply-chain attacks such as Magecart by detecting unhardened headers and default configurations completely automatically.

5.3.2 PCI-DSS Compliance

For e-commerce merchants, PCI-DSS is a compulsory guideline. MisconfigLabs reporting engine is made specifically to help in audit of **PCI-DSS v4.0**. To illustrate, the scanner can assist in achieving the demands of merchants to protect the stored cardholder data by highlighting which cookies have no Secure or HttpOnly flags.

5.3.3 Road to ISO/IEC 27001 Certification

One of the objectives that the MisconfigLab platform will aim to achieve in the future is to assist organizations to become ISO/IEC 27001 qualified. The ISO 27001 involves systematic management of sensitive company data. MisconfigLab fits very well with Annex A.12.6 (Technical Vulnerability Management). Through the constant scans carried out on the platform and documented, an organization can offer evidence of a proactive process of identification of technical weak points to the ISO auditors.

5.4 System Limitations and Critical appraisal

Current Limitations of MisconfigLab:

1. **Lack of Authenticated Scanning**: MisconfigLab operates exclusively as an external threat agent, making it highly effective for perimeter defence assessment. However, its unauthenticated nature prevents it from analysing misconfigurations or vulnerabilities that exist behind a user-role authentication wall, such as those related to internal user roles.

2. **Static Analysis Only (No Dynamic Analysis)**: The scanner currently relies solely on Static Analysis. It cannot capture dynamic, post-rendering configurations, especially in modern security scripts developed with frameworks like React or Angular, which are activated upon user interaction. This is due to the current absence of a "Headless Browser" engine (like Selenium or Playwright).

3. **Directory Brute-Forcing Constraints**: While the directory brute-forcing module is precise, its effectiveness is directly limited by the size of its wordlists. Furthermore, using overly large dictionaries risks triggering highly sensitive Web Application Firewalls (WAFs), which may prevent the scanner's IP address from continuing the process.

5.5 Socio-Technical Impact and Global Retail Resilience

MisconfigLab implementation is a change in digital trust establishment. This platform helps to fill a massive gap in the e-commerce ecosystem as it reduces the entry barrier to professional-level security testing.

It also significantly alters how digital trust is established. This platform addresses a critical void within the e-commerce landscape by lowering the barrier to entry for professional-grade security testing.5.5.1 SME Security Gap Small-to-medium enterprises (SMEs) frequently have low security budgets and are major victims of automated skimming syndicates. MisconfigLab offers such merchants the democratized security tool, which enables them to keep the defensive stance on par with bigger players.

This stability is a key to worldwide digital economy trust, with one of the smallest vendors being frequently attributed as a point of access to bigger chains.

5.5.1 Sustainability of Security Operations

High false-positive rates and significant manual reporting demands often lead to analyst "burnout" within security operations. MisconfigLab directly addresses this operational risk and promotes sustainability through automation. This automation minimizes noise and streamlines compliance mapping, aligning with the ISO 27001 standard's emphasis on efficient and reliable security processes.

5.6 Future Implementation Roadmap

The proposed three-stage roadmap will be needed to transform MisconfigLab into an industry leader:

5.6.1 Phase 1 Venerable AI Integration

The next would be the transition between Rules and Intelligence. The next generation will be based on using Machine Learning (ML) to detect unusual setups. The scanner can induce screening behaviour as suspicious even when this conduct does not violate a predefined rule by educating a model on thousands of instances of secure and compromised websites, which is crucial in identifying zero-day-old Magecart payloads.

5.6.2 Phase 2: Automatic remediation (Auto-Fix)

Identification is not the greatest bottleneck to security, but remediation. We are to implement an Auto-remediation Module. When the scanner detects the missing header, it will create the specific code fragments needed by a user on a particular web server (Nginx or Apache) and provide an option to push that fix, through a GitHub Pull Request, dropping Time-to-Fix timeframes for days to seconds.

5.6.3 Phase 3: API and DevSecOps Integration of Enterprise

We would like to develop a Full REST API to serve big-box stores. This would enable companies to install MisconfigLab in their CI/CD pipelines. Whenever a developer commits some new code, MisconfigLab would run in code scanner mode. Once the Transaction Safety Score dropped below a given threshold (e.g. 80/100), the deployment was halted.

Bibliography / References

Literature Review

[1] Capital One Shopping Research, "Ecommerce Statistics 2024," 2024. [Online]. Available: https://capitaloneshopping.com/research/ecommerce-statistics/

[2] Ciber 4 All Team, "Web Vulnerability Scanner Limitations," May 2024. [Online]. Available: https://www.tarlogic.com

[3] eMarketer, "Ecommerce Accounts for More Than 20% of Worldwide Retail Sales Despite Slowdown," 2024. [Online]. Available: https://www.emarketer.com

[4] e-SPIN Group, "Automated Web Vulnerability Scanners Limitations," Mar. 2025. [Online]. Available: https://www.e-spincorp.com

[5] Fudo Security, "10 Lessons from 2024 Big Data Breaches," Jan. 2025. [Online]. Available: https://www.fudosecurity.com

[6] Kiteworks, "Top 11 Data Breaches and Their Risk Exposure Score," Apr. 2025. [Online]. Available: https://www.kiteworks.com

[7] L. Molnar, "The Illusion of Automated Vulnerability Scans," Jul. 2023. [Online]. Available: https://blog.hckrt.com

[8] OWASP Foundation, "OWASP Top 10:2021 – A05:2021 Security Misconfiguration," 2021. [Online]. Available: https://owasp.org

[9] PCI Security Standards Council, "APAC E-commerce Threat Trends and PCI DSS v4.0," Oct. 2023. [Online]. Available: https://www.pcisecuritystandards.org

[10] Python Software Foundation, "asyncio — Asynchronous I/O," Python 3.10.13 documentation, 2023. [Online]. Available: https://docs.python.org/3/library/asyncio.html

[11] Red Stag Fulfillment, "What Share of Global Retail Sales Is Ecommerce?," 2024. [Online]. Available: https://redstagfulfillment.com

[12] P. Roychowdhury, "Misconfiguration Attacks: Real-World Examples Key Takeaways," Sep. 2025. [Online]. Available: https://appaxon.ai

[13] TuraColabs, "Annual eCommerce ThreatScape 2024," 2024. [Online]. Available: https://www.turacolabs.com/ecommerce-threatscape

Articles for Research and Learning

IBM Security, *Cost of a Data Breach Report 2024*. IBM, Jul. 2024. Available: https://www.ibm.com/reports/data-breach. Accessed: Apr. 13, 2026.

IBM Security, *X-Force Threat Intelligence Index 2026*. IBM Corporation, Feb. 2024. Available: https://www.ibm.com/reports/threat-intelligence. Accessed: Apr. 13, 2026.

PCI Security Standards Council, *Payment Card Industry (PCI) Data Security Standard v4.0.1*. PCI SSC Document Library, 2024. Available: https://www.pcisecuritystandards.org/document_library/. Accessed: Apr. 13, 2026.

Holm Security, “PCI DSS 4.0 - New Standards That Could Shut-Off Your E-Commerce Business,” *Holm Security Resources*, 2024. Available: https://www.holmsecurity.com/resources/business-risks-pci-dss-4. Accessed: Apr. 13, 2026.

Cycode, “Application Vulnerability Scanning: Tools & Guide,” *Cycode Blog*, 2024. Available: https://cycode.com/blog/application-vulnerability-scanning/. Accessed: Apr. 13, 2026.

Vectra AI, “Vulnerability scanning: types, process, and compliance guide,” 2024. Available: https://www.vectra.ai/topics/vulnerability-scanning. Accessed: Apr. 13, 2026.

Palo Alto Networks, “What is Security Misconfiguration? API Security Risks,” *Cyberpedia*, 2023. Available: https://www.paloaltonetworks.com/cyberpedia/security-misconfiguration-api8. Accessed: Apr. 13, 2026.

Santander Cyber Security Research, “Semgrep Rules for Detecting Magecart Skimmers and Obfuscated JavaScript,” GitHub, 2025. Available: https://github.com/Santandersecurityresearch/e-Skimming-Detection. Accessed: Apr. 13, 2026.

S. Badotra and A. Sundas, “A systematic review on security of E-commerce systems,” *International Journal of Applied Science and Engineering*, vol. 18, no. 2, 2021, Art. no. 2020323. Available: https://gigvvy.com/journals/ijase/articles/ijase-202106-18-2-010.pdf. Accessed: Apr. 13, 2026.

Visualping, “What is Magecart? How E-Commerce Web Skimming Attacks Work,” *Visualping Blog*, Jan. 2026. Available: https://visualping.io/blog/what-is-magecart. Accessed: Apr. 13, 2026.

S. Badotra and A. Sundas, "A systematic review on security of E-commerce systems," *International Journal of Applied Science and Engineering*, vol. 18, no. 2, Art. no. 2020323, 2021.Available: https://gigvvy.com/journals/ijase/articles/ijase-202106-18-2-010.pdf. Accessed: Apr. 14, 2026.

K. Abdulghaffar, N. Elmrabit, and M. Yousefi, "Enhancing web application security through automated penetration testing with multiple vulnerability scanners," *Computers*, vol. 12, no. 11, p. 235, 2023. Available: https://doi.org/10.3390/computers12110235

S. Alazmi and D. C. De Leon, "A Systematic Literature Review on the Characteristics and Effectiveness of Web Application Vulnerability Scanners," in *IEEE Access*, vol. 10, pp. 33200-33219, 2022, doi: 10.1109/ACCESS.2022.3161522. Available: https://ieeexplore.ieee.org/document/9739725

Appendix A: Professional, Ethical, Legal, and Social (PELS) Analysis

A.1 Background of PELS in Cybersecurity Development

Automated vulnerability scanners will involve intrinsically an uphill struggle through an intricate web of legal limits and ethical duties. Users of cybersecurity engineering are special in that the tools developed to defend against the attack are the same ones that are used to attack. This appraisal is a critical analysis of the Professional, Ethical, Legal, and Social (PELS) implications of the design and implementation of the MisconfigLab platform, as it allows to align the project with the British Computer Society (BCS) Code of Conduct and international law.

Designing and implementing automated vulnerability scanners presents an inherent challenge due to the complex interplay of legal constraints and ethical responsibilities. Cybersecurity engineers operate in a unique environment where the very tools developed for defence are also those capable of exploitation. This appraisal serves as a critical analysis of the Professional, Ethical, Legal, and Social (PELS) implications arising from the MisconfigLab platform's design and deployment. The goal is to ensure the project adheres to the British Computer Society (BCS) Code of Conduct and relevant international legislation.

A.2 Legal Environment and Compliance with the regulations

The development and implementation of MisconfigLab is primarily constrained by the UK's Computer Misuse Act 1990 (CMA). Specifically, this legal framework imposes limitations through Section 1, which criminalizes unauthorized access to computer material, and Section 3, which prohibits unauthorized actions intended to impair a computer's functioning.

To mitigate the risk of serious legal breaches during the project lifecycle, an "Authorized Targets Only" policy was strictly enforced. The scanning engine was never tested on live, open e-commerce infrastructure. Instead, problem validation was confined to localized testing (detailed in Chapter 4) using single, locally hosted Docker containers (Targets Alpha, Beta, and Gamma).

If MisconfigLab were to be developed into a commercial Software-as-a-Service (SaaS) product, legal liability would mandate the implementation of robust "Proof of Ownership" measures. This would require a cryptographic token (such as a DNS TXT record or an HTML file in the root directory) to be deployed by a client. This token would need to be in place before the target domain could be scanned, thereby mathematically verifying the user's legal authorization to audit that domain.

A.3 The Ethical "Dual-Use" Dilemma

One of the issues in this area of research that raises a serious ethical dilemma is the so-called Dual-Use dilemma. MisconfigLab is meant to detect disaster security misconfigurations in such a way that system administrators can rectify them. But when it gets into the wrong hands of an unrecognized threat actor, the same platform can aid reconnaissance efforts in a way that can be used to select the weak targets to exploit as soon as possible.

One feature of the platform is the explicit exclusion of active exploitation modules; it will discover an open directory, but it is not the scripts required to adjacently steal the data contained in it. Moreover, the implementation of the role-based access control (RBAC) allows to make sure that the tool cannot be used anonymously. The platform is accountable by giving all scan activity an authenticated and logged session, thus discouraging its use as an anonymous tool among cyber-criminals.

A.4 Social Impact: Addressing the "Security Poverty Line"

The social strategy of the project is based on the need to respond to what the experts in the industry have called the Security Poverty line. The contemporary systems of cybersecurity defence are out of budget. Big box retailers hire special DevSecOps teams, and buy costly enterprise scanners, which are well above the poverty line. On the other hand, quite frequently, Small-to-Medium Enterprises (SMEs) that form most online merchants are operating below this line and cannot afford dedicated security tooling.

The social effect of this economic difference is that it directly impacts the end-consumer. The enjoyment of identity theft, credit card fraud, and the resultant psychological and financial torment fall on the everyday consumer when an SME is being compromised by a Magecart attack owing to a miss Content-Security-Policy.

MisconfigLab plays an important social role by opening high-level security intelligence to the masses. The platform reduces the entry barrier in terms of financial and technical resources by using lightweight, open-source technologies (Python, Flask) and automation of the task of translating the complex metrics of the CVSS into an easily readable, accessible document, namely, the Transaction Safety Score. This will enable the smaller merchants to defend their customers leading to the creation of a safer and more resilient digital society.

A.5 Data Protection and GDPR Compliance

To evolve MisconfigLab from an academic prototype into a commercial Software as a Service (SaaS) application, mandatory compliance with the General Data Protection Regulation (GDPR) will be required. This necessity stems from the platform's intrinsic function of processing and storing extremely sensitive vulnerability intelligence.

The current persistent storage of scan results, which may encompass sensitive data such as exposed API keys or infrastructure details, utilizes an SQLite database. To ensure compliance with the General Data Protection Regulation's (GDPR) '**Right to Erasure**' (Article 17) and adhere to data minimization principles, the production version necessitates the implementation of an automated data lifecycle management policy. This policy must incorporate a user-accessible "**Wipe Client History**" function and an automated cryptographic shredding protocol, ensuring the complete and irreversible purging of all scan data after a period of 30 days.

Furthermore, the platform utilizes dynamic session cookies to enforce its Role-Based Access Control (RBAC) architecture. While these are classified as 'strictly necessary' under the Privacy and Electronic Communications Regulations (PECR) and **do not legally require an explicit "Accept Cookies"** banner, the platform must still provide a transparent Privacy Policy detailing how this session data is handled and ensuring it is never used for third-party tracking.

Appendix B: Meeting Logs

Date	**Topics Discussed**	**Action Items / Next Steps**
Week 2 (Feb 2)	Discussed OWASP A05 focus, project aims, and initial research directions.	Began literature review and research process.
Week 3 (Feb 9)	Reviewed, Literature review and initial prototype of how the website should be.	Begin planning flask and SQLite routes.
Week 4 (Feb 16)	Demonstrated first complete draft of Literature review on latex.	Researched on various articles to gather and add more information
Week 6 (Mar 2)	Completed Literature review and submitted with full focus going to the completion of the website platform	Began working on Website's key templates and routing the backend
Week 8 (Mar 16)	Built a fully function general scanner platform yet with more e-commerce information being yet to implement.	Report progress halfway completed with the code needing some final changes
Week 10 (Apr 2)	Completion with the implementation of a full function E-commerce web vulnerability scanner platform.	Full focus on the completion of the report.

Appendix C: Ethics Form

Middlesex University Dubai

Research Ethics Screening Form for Students

Middlesex University is concerned with protecting the rights, health, safety, dignity, and privacy of its research participants. It is also concerned with protecting the health, safety, rights, and academic freedom of its students and with safeguarding its own reputation for conducting high quality, ethical research.

This Research Ethics Screening Form will enable students to self-assess and determine whether the research requires ethical review and approval before commencing the study.

Student Name: Deepak Prabaharan		Email: DP983@live.mdx.ac.uk
	Research project title: Website Security Misconfiguration Detection Platform	
	Programme of study/module: CST 3590 Individual Project	
Supervisor Name: Shafa Fathima Jaffar	Siddique	Email: s.siddique@mdx.ac.ae

Please answer the following questions to determine whether your proposed activity requires ethical review and approval		
1. Will the research 'involve human participants,' with or without their knowledge or consent? *('Human participants' is a wide phrase including, but not limited to, observation, questionnaires, interviews (online and hard-copy), focus groups, social media platforms, etc., visual recordings (e.g., photos, video), audio recordings (e.g., digital, tape), or other human data/materials (e.g., blood, saliva, tissues or other human samples). It also includes yourself in cases where you, as the researcher, are planning to conduct research on yourself or to be involved in the same way as other participants in the project)*	☐ Yes	☒ No
2. Will the research involve animals or animal parts?	☐ Yes	☒ No
3. Will the research involve any activity that might cause damage or present a significant risk to society? *(e.g., to precious artefacts or the environment)*	☐ Yes	☒ No
4. Is the research likely to put you or others to any risks other than considered everyday risks? *(e.g., risk of physical or psychological harm, engagement in illegal activities, working in a foreign country, travel risks, working alone, breaching security systems or searching the internet for data about highly sensitive topics such as sexual abuse, terrorism, etc.)*	☐ Yes	☒ No
5. Will the research include digital information/data from the internet, social media platforms, Apps, or smart devices with or without users' knowledge or consent, and/or could it lead to users being identified?	☐ Yes	☒ No
6. Will the research require approval to access any data? *(e.g., access data through individuals and/or data through an external organisation(s))*	☐ Yes	☒ No
7. Could anyone involved in the research have a potential conflict of interest or lack of impartiality?	☐ Yes	☒ No
8. Will your project involve working with any substances and/or equipment that may be considered hazardous to you or others?	☐ Yes	☒ No
9. Will the research involve discussion of sensitive topics? *(e.g., sexual activity, drug use, national security etc.)*	☐ Yes	☒ No
10. Will the outputs from your research (e.g., products, reports, publications, etc.) likely cause any harm to you, others, or to society; or have legal issues?	☐ Yes	☒ No

Middlesex University Dubai

If you have answered 'Yes' to ANY of the above questions, your application requires ethical review and approval prior to commencing your research. Please complete the 'Application for Ethical Approval for Research Projects for Students' form

If you have answered 'No' to ALL of the above questions, your application may not require ethical review and approval before commencing your research. Your research supervisor will confirm this below.

Student Signature: Date:

Deepak Haran 31/1/2026

To be completed by the supervisor:

Based on the details provided in the self-assesment form, I confirm that:	
The study does not require ethical review and approval	☐
The study requires ethical review and approval	yes ☐

Superivsor Signature:....S.S..... Date:..2/3/2026...

Appendix D: Customer Journey Map (CJM)

To adequately close the critical communication divide between granular, technical vulnerability data and high-level executive stakeholders, the MisconfigLab platform used a design solution guided by a rigorous, user-centric design based on a detailed Customer Journey Map (CJM). It was used as the blueprint of this strategic artifact, carefully drawing the end-to-end user experience and providing a smooth socio-technical alignment.

Strategic Alignment using Customer Journey Map (CJM).

The main task of the CJM was to trace the emotional state of the user, strategic business goals, and business pain points directly to the advanced processes and architectural capabilities of the platform. This mapping was essential in converting the technical operations into a frictionless, enterprise level user experience.

Specific architecture was developed to help meet the special constraints of Small-to-Medium Enterprises (SMEs) below the security poverty line. The main architectural elements that were directly mapped to these operational pain points:

Asynchronous Scanning Thread: This element is a direct response to the resource limitations and requirement of the SME to have security monitoring that is non-disruptive and continuous. The platform uses scans in the background to make sure the vulnerability discovery and data processing do not affect the performance and availability of the key business systems. This is seen by the user as zero operational overhead and always-on security.

Role-Based Access Control (RBAC) Data Isolation: This is the most important feature in terms of dealing with governance and compliance issues. MisconfigLab will be able to guarantee that sensitive vulnerability data are compartmentalized and can only be accessed by authorized personnel by executing strict RBAC. This data isolation which is systematic in nature will not only increase security but also directly meet the requirement of clear accountability and simplified auditing procedures.

The Culmination: Data to Actionable Compliance.

By aligning the workflow of the user with the capabilities of the system, MisconfigLab guarantees that the process that the user is guided through leads to actionable compliance reporting. The platform capability to directly convert raw technical observations like misconfigurations found by the asynchronous thread into straightforward business-contextualized reports that enables executive stakeholders to make sound risk management choices without necessarily having to dig deep into technical information. This design philosophy is useful in effectively converting technical vulnerability information into a manageable strategic business risk measure, instead of an overwhelming IT issue.

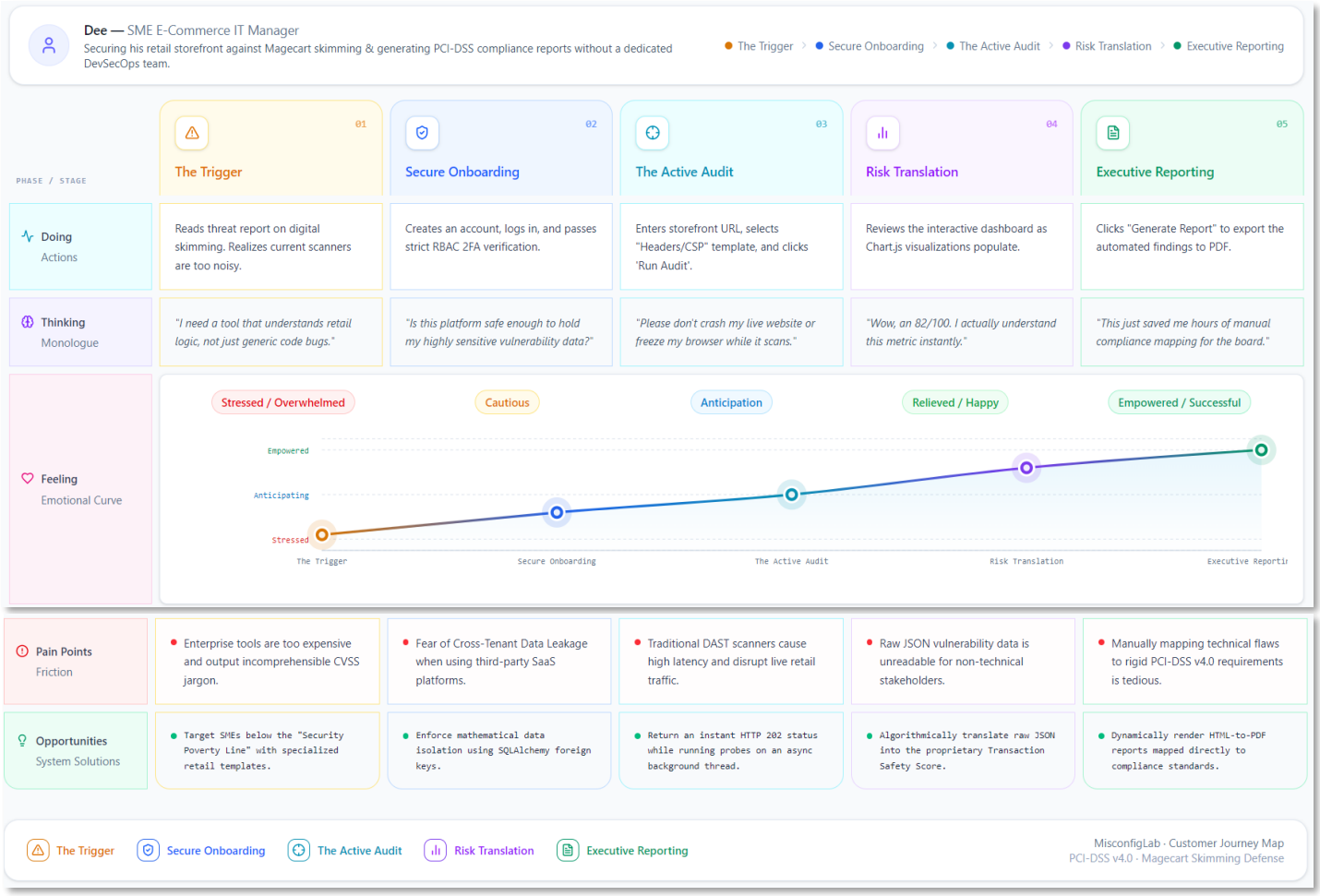


Figure 14: Customer Journey Map detailing the socio-technical interaction between an SME retail manager and the MisconfigLab platform, mapping user intent to specific backend architectural processes.